CSP-203: Software Tools & Technologies Lab



Chapter – 7

Containerization with Docker

Dr. Anand M. Baswade
Indian Institute of Technology, Bhilai
anand@iitbhilai.ac.in

Contents

- ❑ Introduction to Docker
- ❑ Container
- ❑ Docker vs Virtual Machine
- ❑ Build, Ship, Run
- ❑ Docker Images
- ❑ Docker Engine
- ❑ Docker Files

Source: Solomon Hykes's PPT, Founder, Docker Inc.

The Challenge

Multiplicity of Stacks

Static website

nginx 1.5 + modsecurity + openssl + bootstrap 2

User DB

postgresql + pgv8 + v8

Queue

Redis + redis-sentinel

Analytics DB

hadoop + hive + thrift + OpenJDK

Background workers

Python 3.0 + celery + pyredis + libcurl + ffmpeg + libopencv + nodejs + phantomjs

Web frontend

Ruby + Rails + sass + Unicorn

API endpoint

Python 2.7 + Flask + pyredis + celery + psycopg + postgresql-client

Do services and apps interact appropriately?

Multiplicity of hardware environments

Development VM

QA server

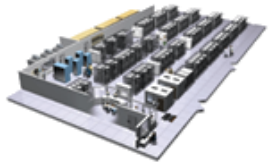
Customer Data Center



Public Cloud

Disaster recovery

Production Servers



Production Cluster



Contributor's laptop



Can I migrate smoothly and quickly?

Another Matrix From Hell

	Development VM	QA Server	Single Prod Server	Onsite Cluster	Public Cloud	Contributor's laptop	Customer Servers
Static website	?	?	?	?	?	?	?
Web frontend	?	?	?	?	?	?	?
Background workers	?	?	?	?	?	?	?
User DB	?	?	?	?	?	?	?
Analytics DB	?	?	?	?	?	?	?
Queue	?	?	?	?	?	?	?



Cargo Transport Pre-1960

Multiplicity of Goods

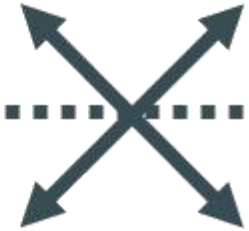








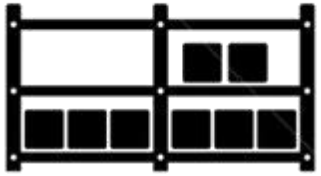
Do I worry about how goods interact (e.g. coffee beans next to spices)



Multipilicity of methods for transporting/storing






Can I transport quickly and smoothly (e.g. from boat to train to truck)

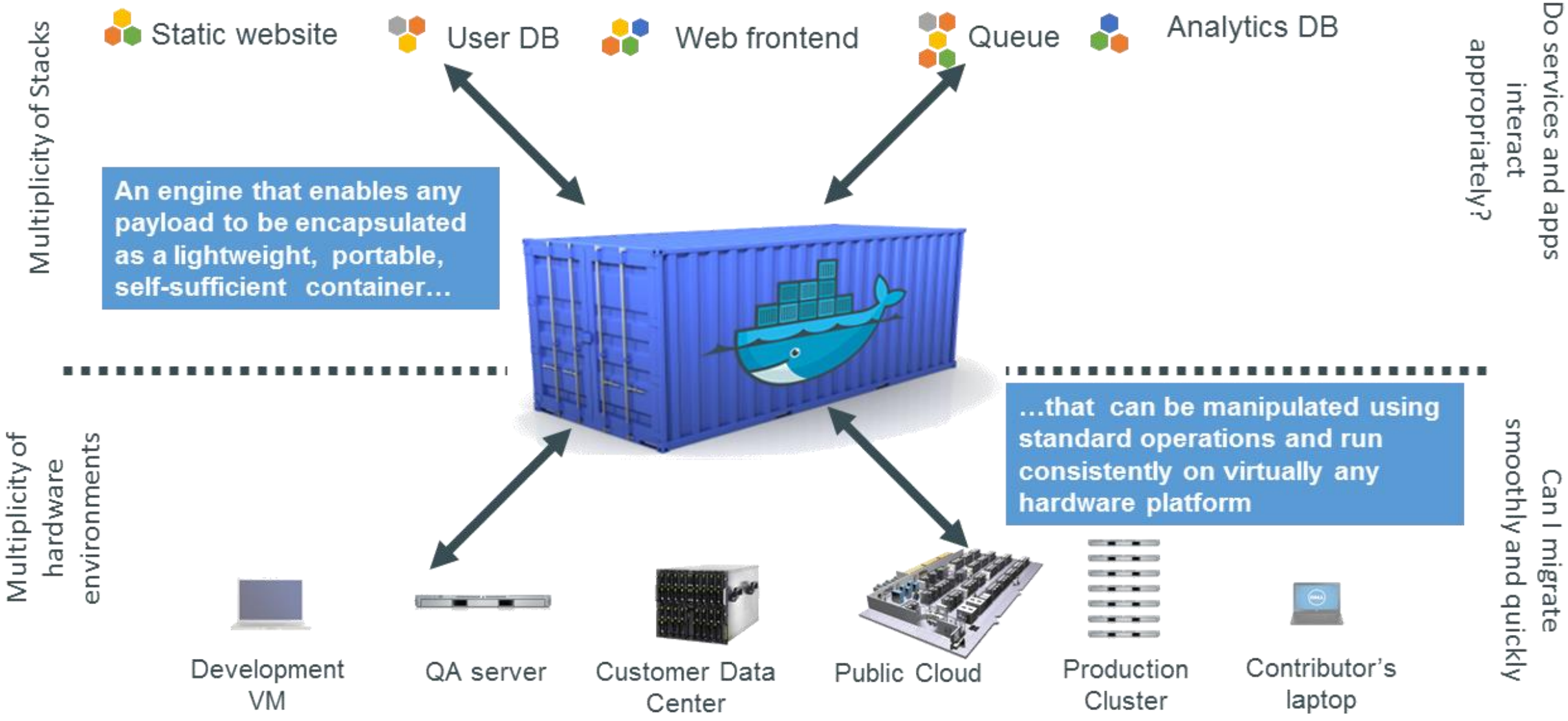
Another Matrix From Hell

	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
							

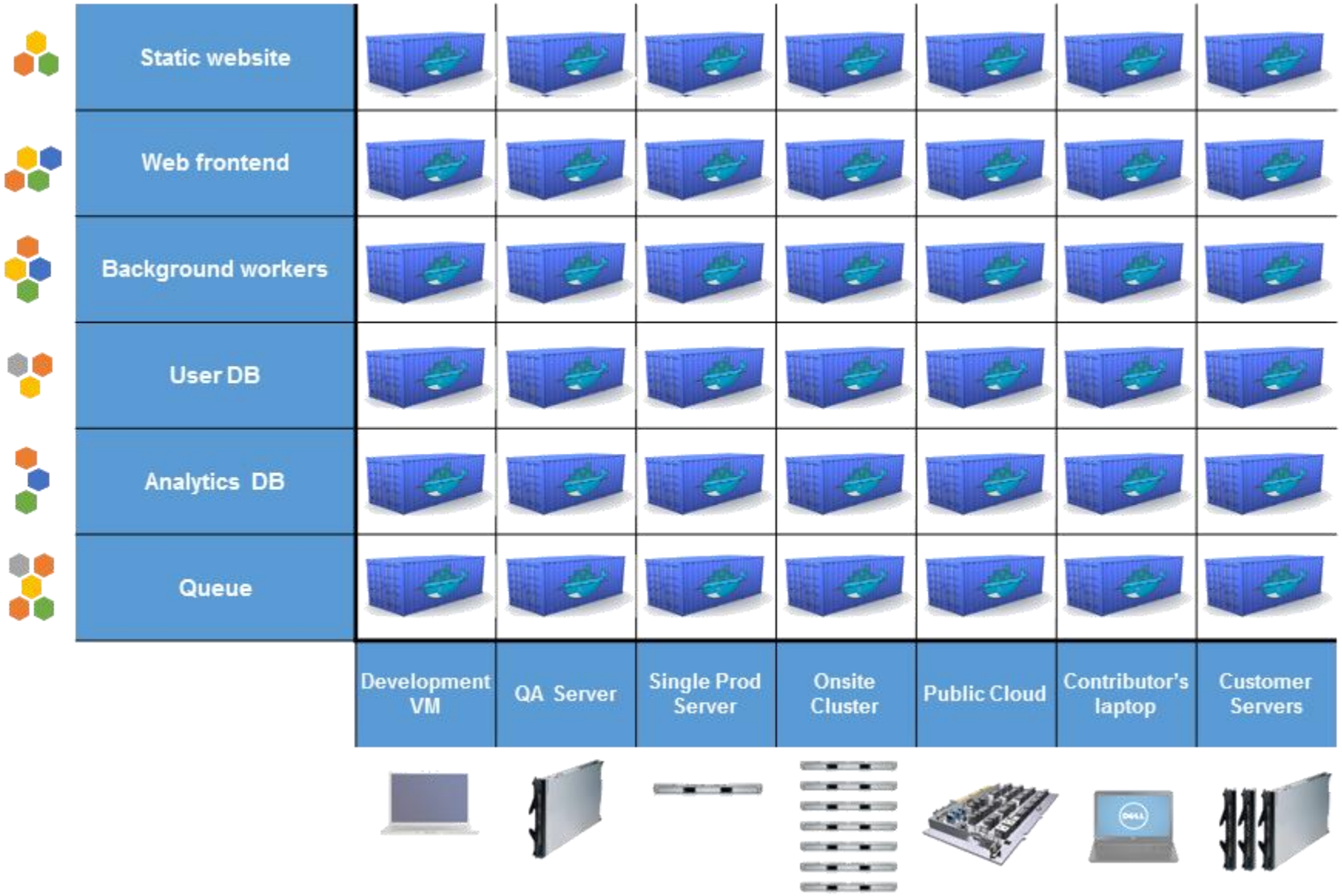
Solution: Intermodal Shipping Container

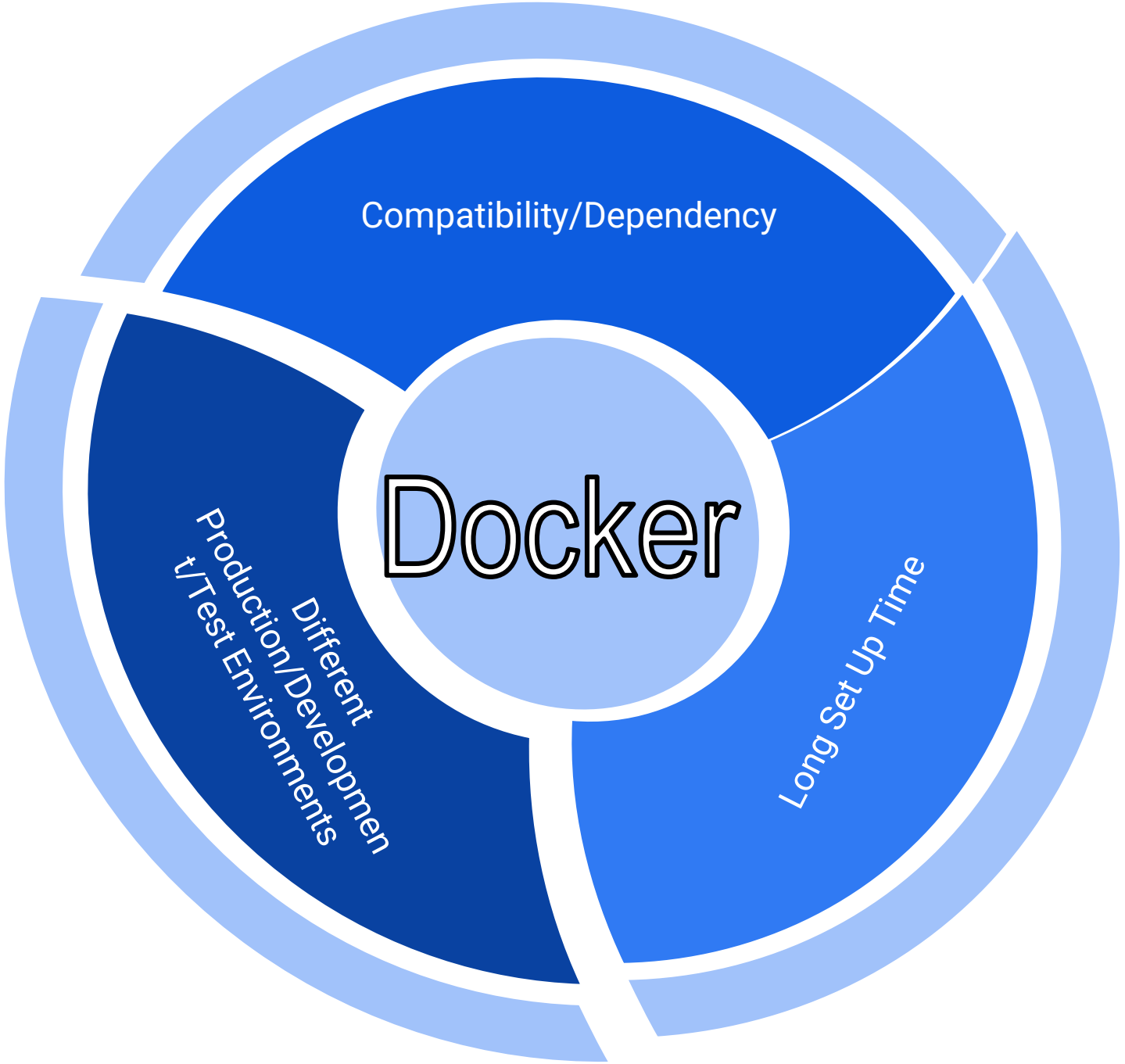


Docker is a Container System for Code



Docker Eliminates the Matrix of Hell




Compatibility/Dependency
Docker
Long Set Up Time
Different Production/Developmen t/Test Environments

What Developers care about?

- Build once... (finally) run anywhere
- A clean, safe, hygienic, portable runtime environment for your app.
- No worries about missing dependencies, packages and other pain points during subsequent deployments.
- Run each app in its own isolated container, so you can run various versions of libraries and other dependencies for each app without worrying.
- Automate testing, integration, packaging...anything you can script.
- Reduce/eliminate concerns about compatibility on different platforms, either your own or your customers.

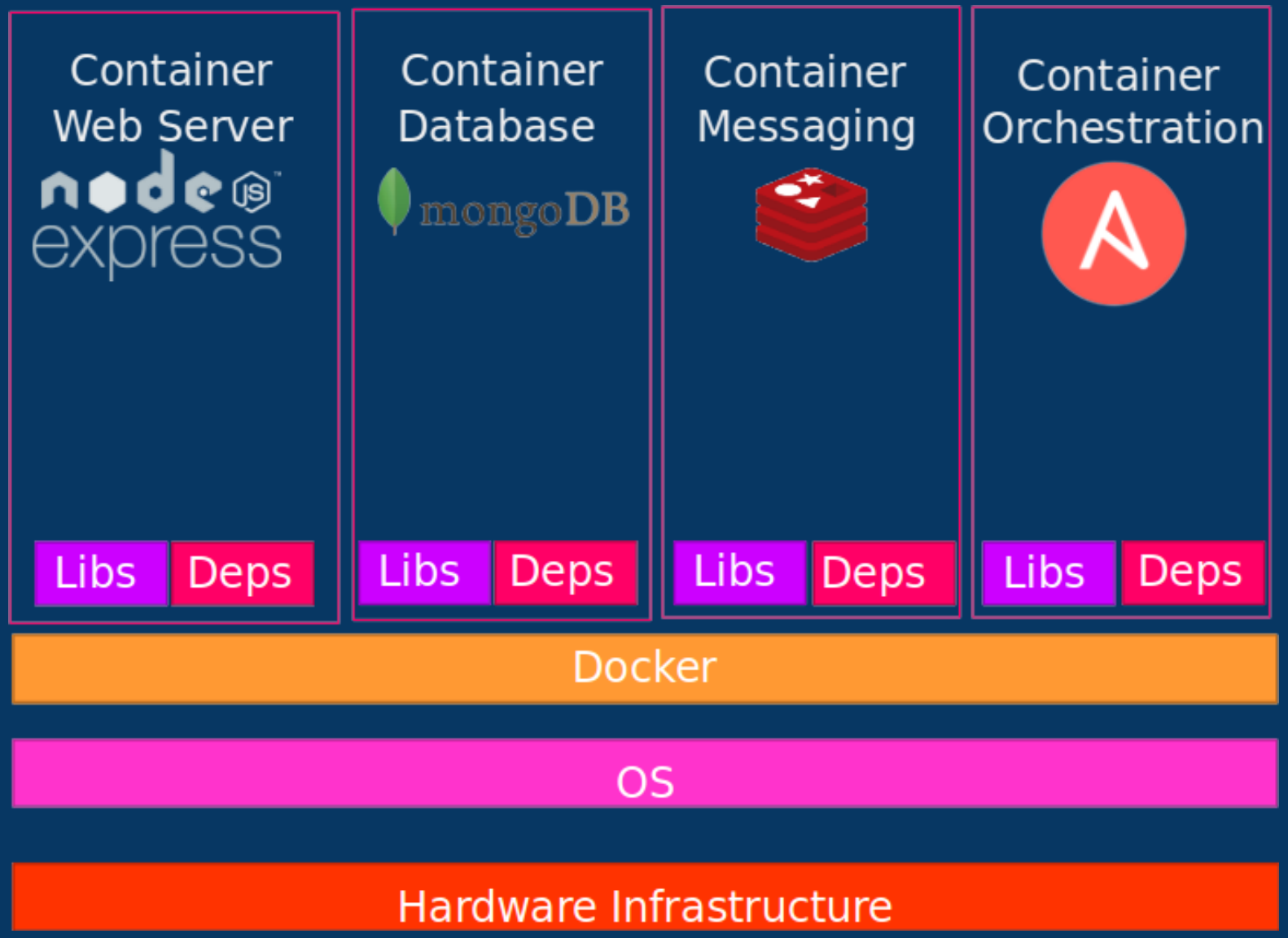
What Administrators care about?

- Configure once... run anything
- Make the entire lifecycle more efficient, consistent, and repeatable
- Increase the quality of code produced by developers.
- Eliminate inconsistencies between development, test, production, and customer environments.
- Support segregation of duties.
- Significantly improves the speed and reliability of continuous deployment and continuous integration systems.
- Because the containers are so lightweight, address significant performance, costs, deployment, and portability issues normally associated with VMs.

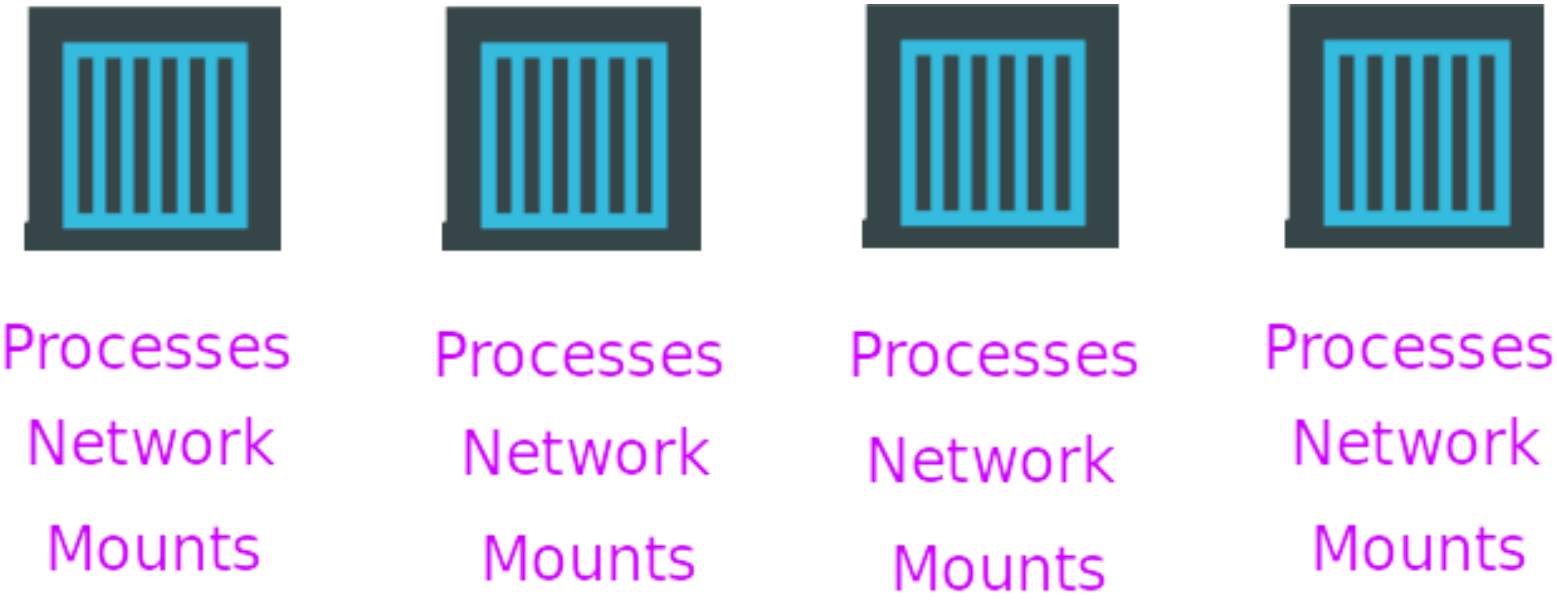


01	**Run everywhere**	• Regardless of kernel version • Regardless of host distro • Physical or virtual, cloud or not • Container and host architecture must match...
02	**Run anything**	• If it can run on the host, it can run in the container • If it can on a Linux kernel, it can run
03	**High level: a lightweight VM**	• Own process space • Own network interface • Can run stuff as root • Can have its own /sbin/init (different from host) • <<machine container>>
04	**Low level: chroot on steroids**	• Can also not have its own /sbin/init • Container = isolated processes • Share kernel with host • <<application container>>

What a Docker Can Do?

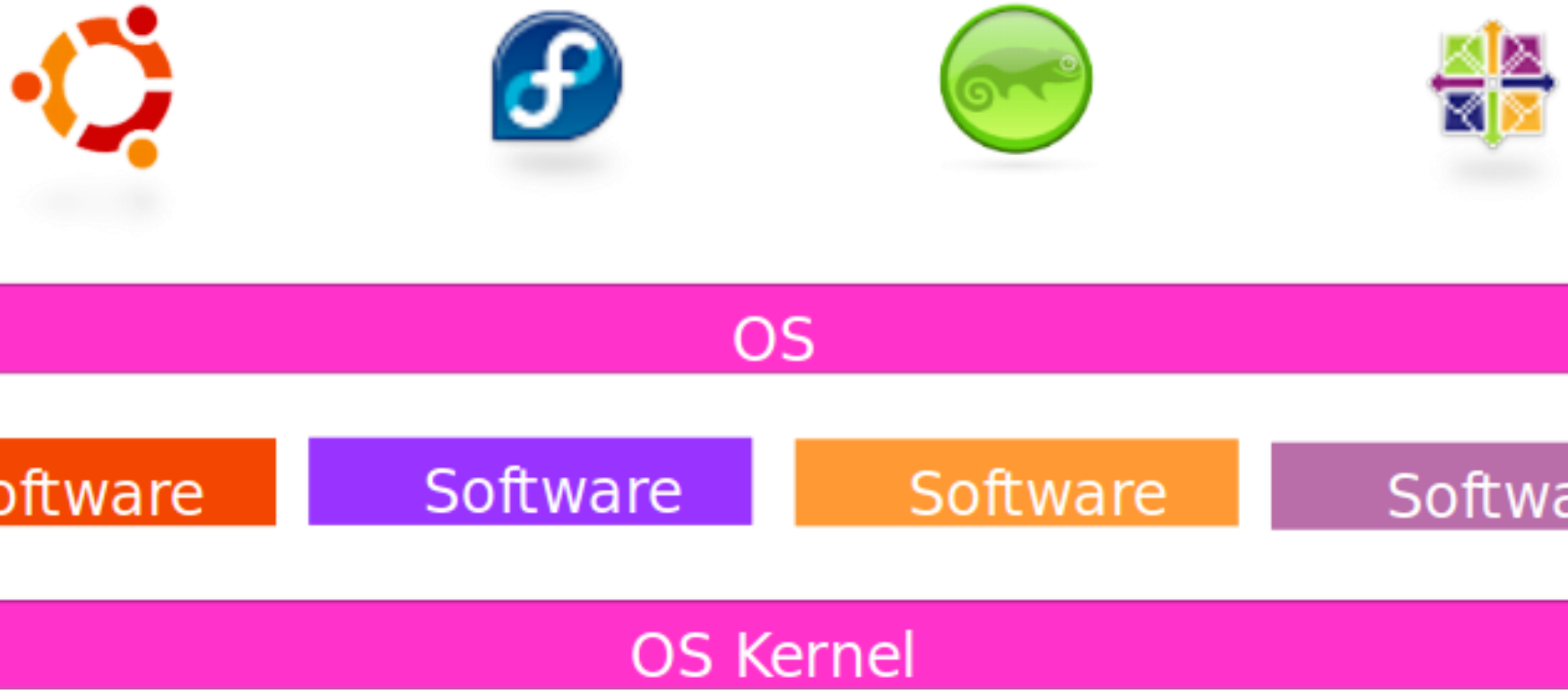


What is Container?

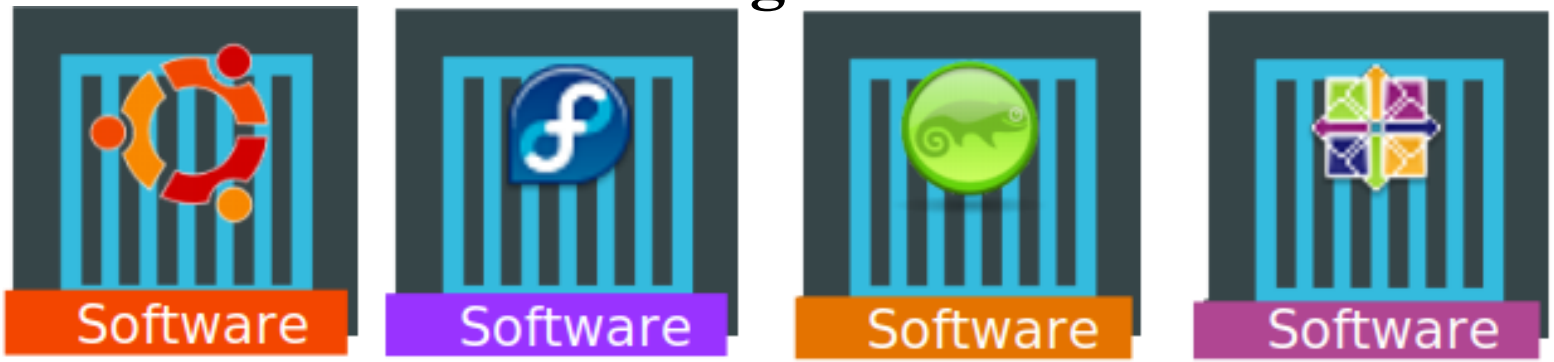


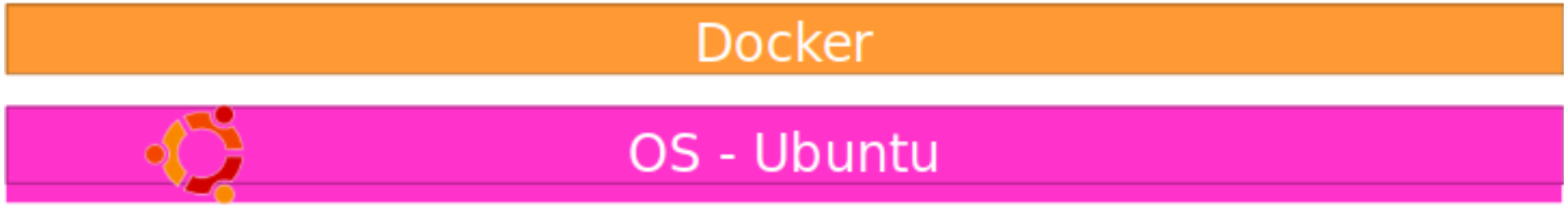
- Standardized packaging for software and dependencies
- Isolate apps from each other
- Share the same OS kernel.
- Works for all major Linux distributions.

Operating System

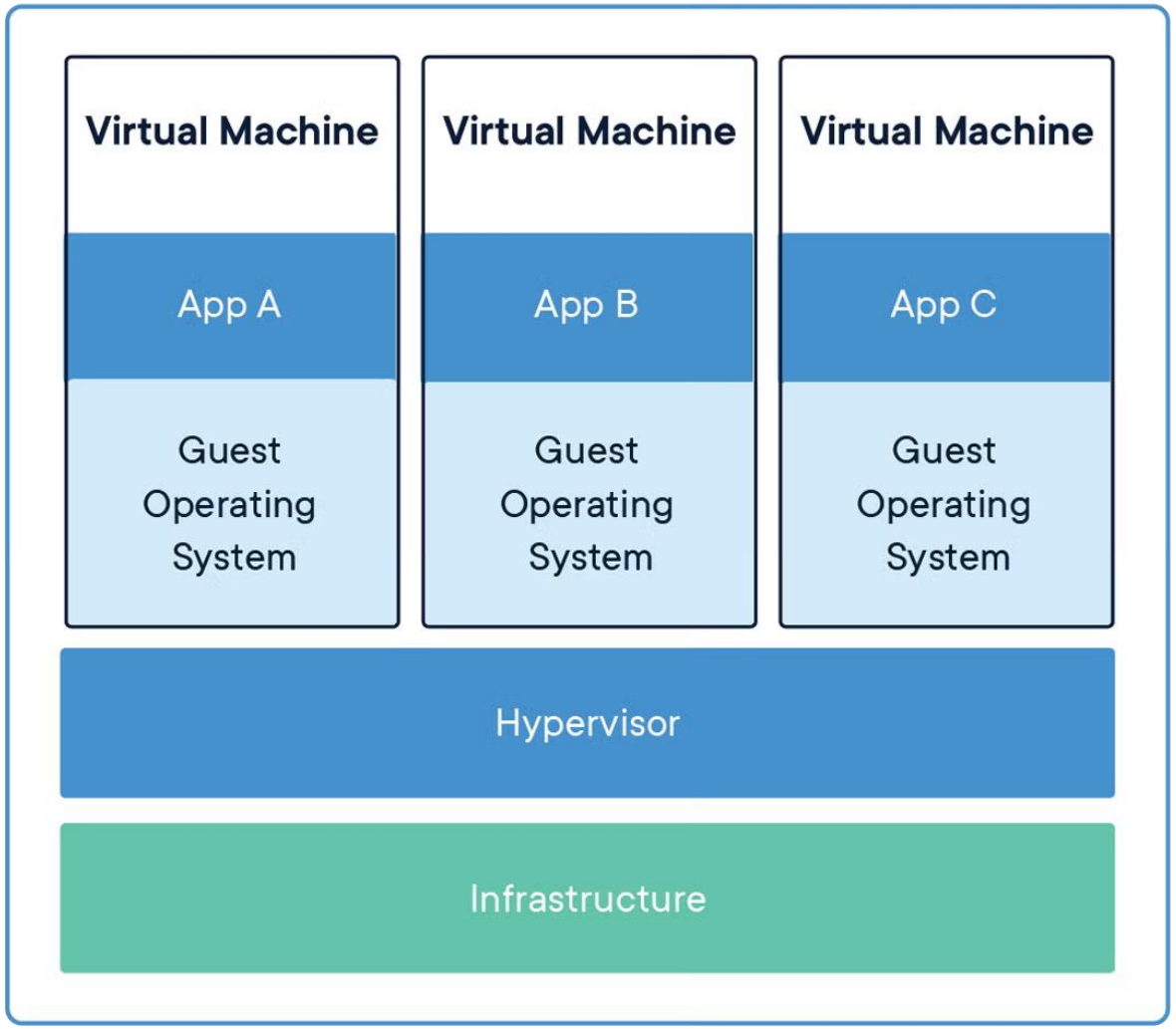


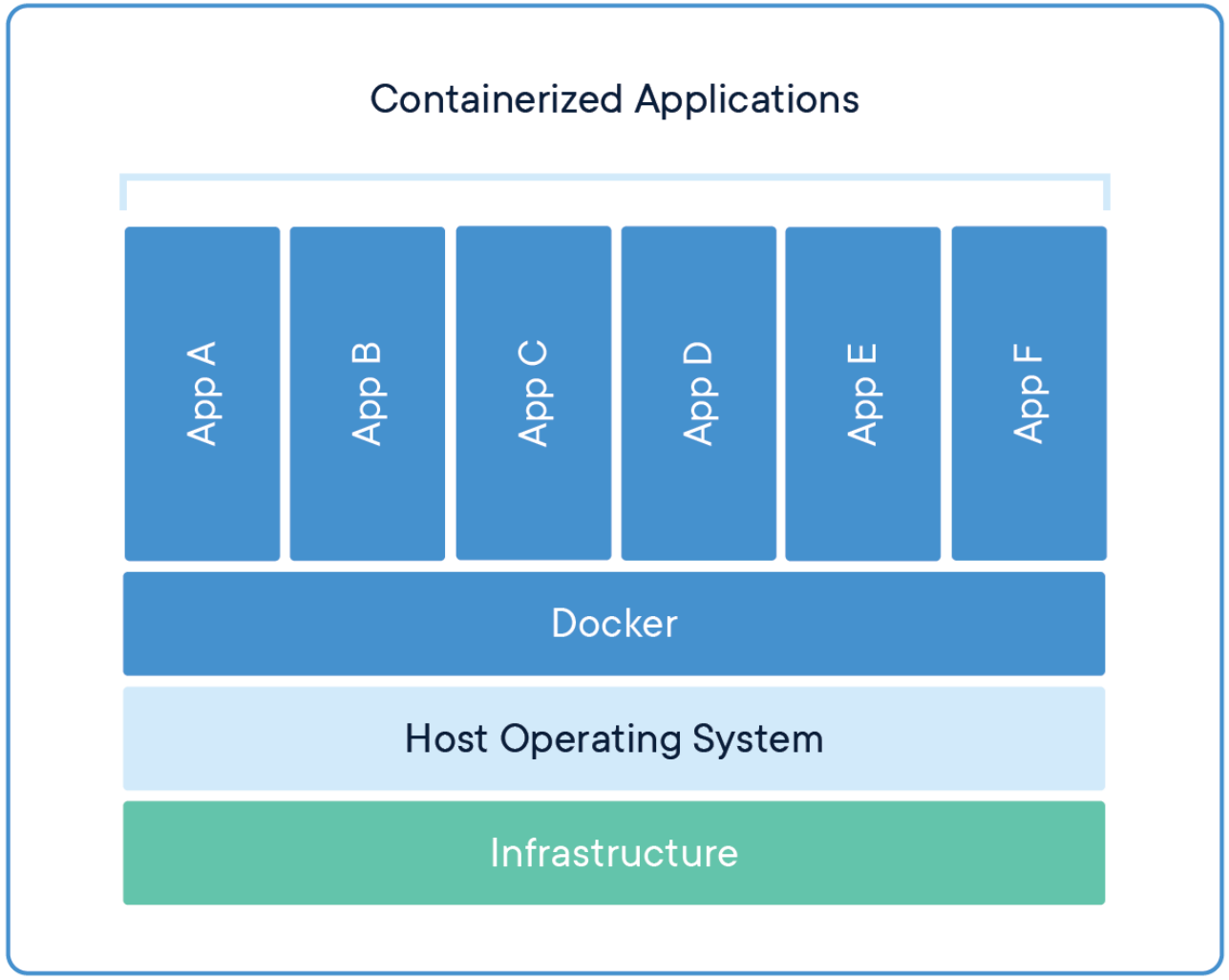
Sharing The Kernel





Docker Vs Virtual Machine





Build, Ship, Run, Any App Anywhere

From Dev

To Ops







Any App



docker

Any OS

Windows

Linux

Anywhere

Physical

Virtual

Cloud

Put it all together: Build, Ship, Run Workflow

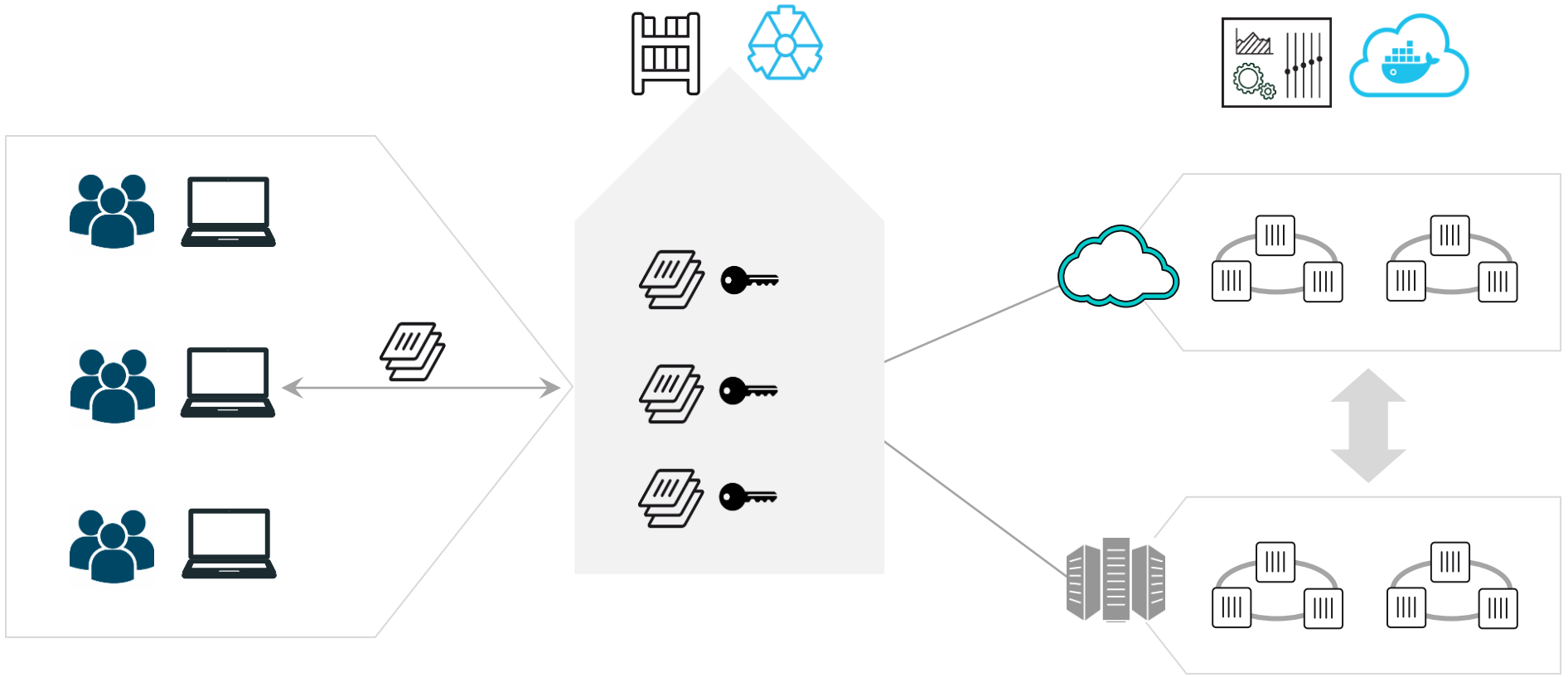
Developers

IT Operations

BUILD
Development Environments

SHIP
Create & Store Images

RUN
Deploy, Manage, Scale



Docker on Linux

- Create a Linux VM (or use physical), and install Docker
  - Requires kernel 3.10
- Stable builds
  - curl –sSL https://get.docker.com/ | sh
- Test and experimental builds
  - curl –sSL https://test.docker.com/ | sh
  - curl –sSL https://experimental.docker.com/ | sh
- Can also manually install (see docs)

Docker for Windows / Mac

- Currently in public beta
- Easy to install: Get up and running on Docker in minutes
- Leverages Hyper-V (Windows) or xhyv (Mac)
  - Docker for Windows requires Windows Pro 10, Enterprise, or Education
- Full API / CLI compatibility
- OS integration for increased stability and speed

Docker for Azure / AWS

- Currently in public beta
  - https://beta.docker.com/
- Easily deploy Docker 1.12 Swarm clusters (Linux)
- Scale up and down easily
- Integrate with underlying platform (i.e. load balancers)

Docker Image

- Standalone executable file.
- Used to create container
- Contains all libraries, dependencies and files that a container need to run.
- Shareable
- Portable
- Similar to Software Binary File

Docker Image Vs Docker Container



Package Template Plan



Docker Container #1



Docker Container #2



Docker Container #3

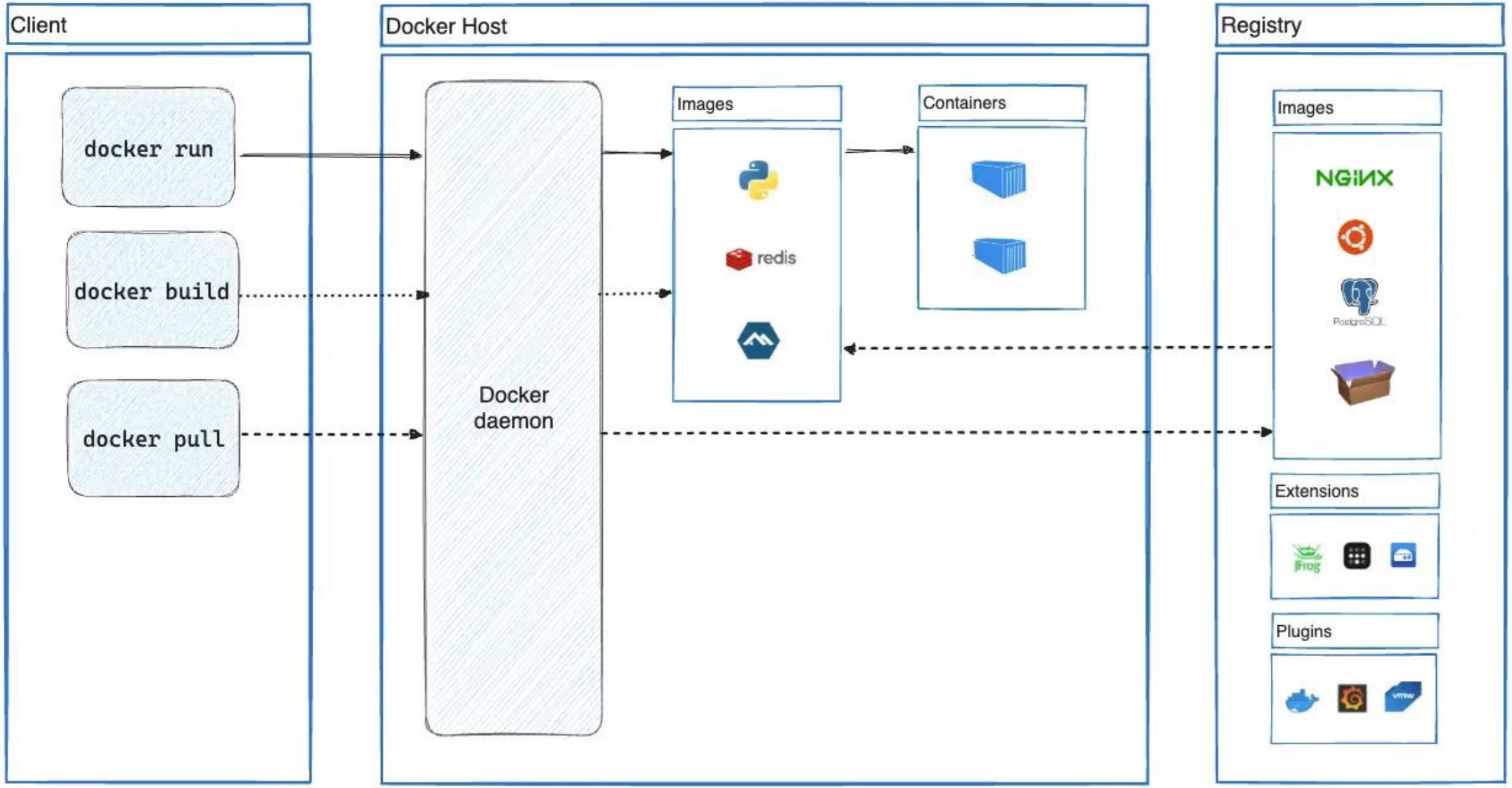
Usage of Docker Image

- That being said, management is easier when you use both images and containers. For example, here's how you can use them together:
  - Scale your application horizontally by running multiple instances of containers based on the same image.
  - Automate continuous integration and deployment (CI/CD) pipelines by using different images for development, testing, and production environments.
  - Tag and manage different versions of your images. This helps you roll back or deploy specific versions as needed.

Docker Engine

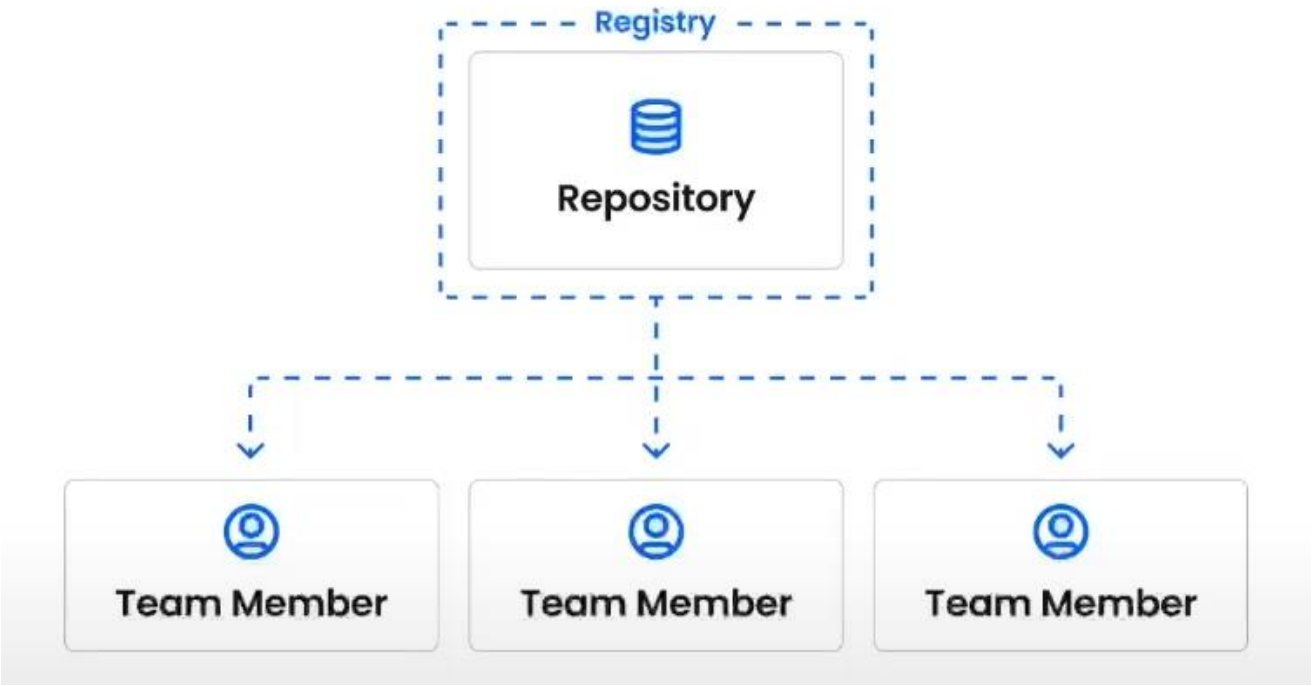
- Docker Engine is an open source containerization technology for building and containerizing your applications. Docker Engine acts as a client-server application with:
  - A server with a long-running daemon process dockerd.
  - APIs which specify interfaces that programs can use to talk to and instruct the Docker daemon.
  - A command line interface (CLI) client docker.

Docker Engine



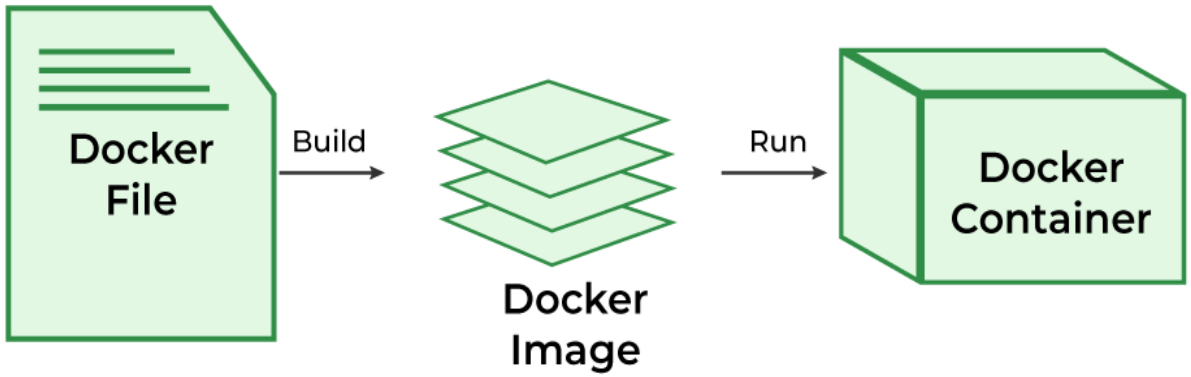
Docker Registry

- A registry contains repositories where your images are stored.
- Allows You to Share Images across teams.
- Options for creating repositories with in a registry:
  - Docker Hub (Default)
  - Amazon ECR
  - Github Container Registry
  - Azure Container Registry
  - Internal Container Registry



Docker Files

- It is a text file.
- Contains all the commands a user could call on the command line to assemble an image.
- Docker can build images automatically by reading the instructions from a Dockerfile.





References:

1. https://docs.docker.com

Next Topic : Lab Session (Docker Setup and Basic Commands)